Module 4: IO Streams and Files

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai Campus

premanand.s@vit.ac.in

April 7, 2026

Module 4: Syllabus Overview

Module Title

IO Streams and Files

Topics Covered:

- File Object
- I/O streams
- FileInputStream & FileOutputStream
- FileReader & FileWriter
- DataInputStream & DataOutputStream
- BufferedInputStream & BufferedOutputStream
- PrintOutputStream
- Serialization & Deserialization

Hook Story: Why Do We Need Files?

Imagine a small clinic or hospital system...

Every day:

- Receptionist enters patient names
- Doctor checks previous prescriptions
- Lab stores reports
- Billing section saves payment details

Question: What happens if the program closes and all data disappears?

Answer: We need permanent storage.

That is where **Files and I/O Streams** come in.

What is I/O?

I/O stands for:

- **Input** $\rightarrow$ Taking data into the program
- **Output** $\rightarrow$ Sending data out of the program

Examples:

- Reading from keyboard
- Reading from file
- Writing marks into a file
- Saving student records

What is a Stream?

A Stream is a flow of data.

Analogy: Water Pipe

- Source → File / Keyboard / Network
- Pipe → Stream
- Destination → Java Program

Two Main Types:

- **Input Stream** → Data comes into the program
- **Output Stream** → Data goes out of the program

Types of Streams

Byte Streams

Used for raw data such as:

- Images
- PDFs
- Audio / Video
- Binary files

Character Streams

Used for text data such as:

- Text files
- Java source files
- Notes / reports

Common Java Stream Classes

Category	**Classes**
Byte Streams	FileInputStream, FileOutputStream
Character Streams	FileReader, FileWriter
Buffered Streams	BufferedInputStream, BufferedOutputStream
Data Streams	DataInputStream, DataOutputStream
Formatted Output	PrintStream
Object Streams	ObjectOutputStream, ObjectInputStream

File Object

What is a File Object?

A `File` object represents:

- A file
- A folder (directory)
- A file path

Important:

- It does **not directly read/write data**
- It is used for **file handling operations**

File Object: Syntax

```
import java.io.File;

File f = new File("sample.txt");
```

Common Uses:

- Create a file
- Check whether file exists
- Get file name / path
- Delete a file
- Create a directory

File Object: Example

```
import java.io.File;
import java.io.IOException;
public class FileDemo {
  public static void main(String[] args) throws
      IOException {
    File f = new File("sample.txt");

    System.out.println("Exists: " + f.exists());
    if (f.createNewFile()) {
      System.out.println("File created");
    } else {
      System.out.println("File already exists");
    }

    System.out.println("File name: " +
        f.getName());
    System.out.println("Path: " +
        f.getAbsolutePath());
  }   }
```

Basic Problem Statements: File Object

1. Create a file named `student.txt`
2. Check whether `marks.txt` exists
3. Display file name and path
4. Create a folder called `JavaLab`
5. Delete a file called `temp.txt`

FileInputStream

What is it?

`FileInputStream` is used to:

- Read data from a file
- Read **byte by byte**

Suitable for:

- Text files (as raw bytes)
- Images
- PDFs
- Binary files

FileInputStream: Syntax

```
FileInputStream fis = new
    FileInputStream("sample.txt");
```

FileInputStream: Example

```
import java.io.File;
import java.io.FileInputStream;
import java.io.FileWriter;
import java.io.IOException;

public class FileInputDemo {
  public static void main(String[] args) throws
      IOException {

    // Step 1: Create file object
    File f = new File("sample.txt");

    // Step 2: Create file if not exists
    if (!f.exists()) {
      f.createNewFile();
      FileWriter fw = new FileWriter(f);
      fw.write("Hello Java File Handling!");
      fw.close();
    }
```

FileInputStream: Example

```
        // Step 3: Read file
        FileInputStream fis = new FileInputStream(f);

        int data;
        while ((data = fis.read()) != -1) {
          System.out.print((char) data);
        }

        fis.close();
    }
}
```

Basic Problem Statements: File Object

1. Create a file named `student.txt`
2. Check whether `marks.txt` exists
3. Display file name and path
4. Create a folder called `JavaLab`
5. Delete a file called `temp.txt`

FileOutputStream

What is it?

`FileOutputStream` is used to:

- Write data into a file
- Write **byte by byte**

FileOutputStream: Syntax

```
FileOutputStream fos = new
    FileOutputStream("sample.txt");
```

FileOutputStream: Example

```
import java.io.FileOutputStream;
import java.io.IOException;

public class FileOutputDemo {
  public static void main(String[] args) throws
      IOException {
    FileOutputStream fos = new
        FileOutputStream("sample.txt");

    String msg = "Welcome to Java IO Streams";
    fos.write(msg.getBytes());

    fos.close();
    System.out.println("Data written
        successfully");
  }
}
```

Append Mode

```
FileOutputStream fos = new
    FileOutputStream("sample.txt", true);
```

Meaning: Add data to existing file instead of replacing it.

Basic Problem Statements: FileOutputStream

1. Write your name into a file
2. Store student marks in a file
3. Append attendance details to an existing file
4. Save today's diary note into a file

FileReader

What is it?

`FileReader` is used to:

- Read **character data**
- Best suited for **text files**

FileReader: Syntax

```
FileReader fr = new FileReader("sample.txt");
```

FileReader: Example

```
import java.io.FileReader;
import java.io.IOException;

public class FileReaderDemo {
  public static void main(String[] args) throws
      IOException {
    FileReader fr = new FileReader("sample.txt");

    int ch;
    while ((ch = fr.read()) != -1) {
      System.out.print((char) ch);
    }

    fr.close();
  }
}
```

Basic Problem Statements: FileReader

1. Read a poem from a file
2. Read file and count words
3. Read file and display uppercase letters only
4. Read student feedback from a file

FileWriter

What is it?

`FileWriter` is used to:

- Write **character data**
- Best suited for **text files**

FileWriter: Syntax

```
FileWriter fw = new FileWriter("sample.txt");
```

FileWriter: Example

```
import java.io.FileWriter;
import java.io.IOException;

public class FileWriterDemo {
  public static void main(String[] args) throws
      IOException {
    FileWriter fw = new FileWriter("sample.txt");

    fw.write("Java FileWriter Example");
    fw.write("\nLearning IO Streams");

    fw.close();
    System.out.println("Written successfully");
  }
}
```

Basic Problem Statements: FileWriter

1. Write student feedback into a file
2. Save class timetable into a file
3. Append daily expenses to an existing file
4. Store a short paragraph in a file

DataInputStream and DataOutputStream

Purpose:

- Used to read/write **primitive data types**

Examples of supported types:

- `int`
- `double`
- `float`
- `boolean`
- `char`
- `String` (using UTF)

Syntax

```
DataOutputStream dos =
new DataOutputStream(new
    FileOutputStream("data.txt"));

DataInputStream dis =
new DataInputStream(new
    FileInputStream("data.txt"));
```

Data Streams: Example

```
import java.io.*;

public class DataStreamDemo {
  public static void main(String[] args) throws
      IOException {

    DataOutputStream dos = new DataOutputStream(
    new FileOutputStream("data.txt"));
    dos.writeInt(101);
    dos.writeUTF("Prem");
    dos.writeDouble(89.5);
    dos.close();
```

Data Streams: Example

```
import java.io.*;

public class DataStreamDemo {
  public static void main(String[] args) throws
      IOException {

    DataOutputStream dos = new DataOutputStream(
    new FileOutputStream("data.txt"));
    dos.writeInt(101);
    dos.writeUTF("Prem");
    dos.writeDouble(89.5);
    dos.close();
```

Important Note

Read order must match write order!

If you write:

- `writeInt()`
- `writeUTF()`
- `writeDouble()`

Then you must read:

- `readInt()`
- `readUTF()`
- `readDouble()`

Basic Problem Statements: Data Streams

1. Store student ID, name, and marks
2. Save employee salary details
3. Store product price and quantity
4. Save exam pass/fail status

BufferedInputStream

What is it?

`BufferedInputStream` reads data **faster** using a memory buffer.

Why needed?

- Normal reading may be slower
- Buffered reading improves efficiency

BufferedInputStream: Syntax

```
BufferedInputStream bis =
new BufferedInputStream(new
    FileInputStream("sample.txt"));
```

BufferedInputStream: Example

```
import java.io.*;

public class BufferedInputDemo {
  public static void main(String[] args) throws
      IOException {
    BufferedInputStream bis =
    new BufferedInputStream(new
        FileInputStream("sample.txt"));

    int data;
    while ((data = bis.read()) != -1) {
      System.out.print((char) data);
    }

    bis.close();
  }
}
```

BufferedOutputStream

What is it?

`BufferedOutputStream` writes data **faster** using a memory buffer.

BufferedOutputStream: Syntax

```
BufferedOutputStream bos =
new BufferedOutputStream(new
    FileOutputStream("sample.txt"));
```

BufferedOutputStream: Example

```
import java.io.*;

public class BufferedOutputDemo {
  public static void main(String[] args) throws
      IOException {
    BufferedOutputStream bos =
    new BufferedOutputStream(new
        FileOutputStream("sample.txt"));

    String msg = "Buffered writing is faster";
    bos.write(msg.getBytes());

    bos.close();
    System.out.println("Data written");
  }
}
```

flush() Method

```
bos.flush();
```

Meaning: Forces buffered data to be written immediately.

Basic Problem Statements: Buffered Streams

1. Read a large file efficiently
2. Write a report using buffered output
3. Copy a file using buffering
4. Save logs using buffered writing

PrintStream

What is it?

`PrintStream` is used to:

- Print data easily
- Write formatted output into a file

Advantage:

- No need for manual conversion

PrintStream: Syntax

```
PrintStream ps = new PrintStream("output.txt");
```

PrintStream: Example

```
import java.io.*;

public class PrintStreamDemo {
  public static void main(String[] args) throws
     IOException {
    PrintStream ps = new
       PrintStream("output.txt");

    ps.println("Student Report");
    ps.println("Name: Prem");
    ps.println("Mark: 95");
    ps.println("Result: Pass");

    ps.close();
    System.out.println("Printed successfully");
  }
}
```

Basic Problem Statements: PrintStream

1. Generate student report card
2. Print bill receipt into a file
3. Store formatted exam result
4. Create an invoice file

Serialization

Hook Question:

If a Java program closes, what happens to the object in memory?

Answer: It disappears.

Serialization means:

Definition

Converting an object into a byte stream so that it can be saved into a file.

Real-Time Applications of Serialization

- Save student records
- Save patient records
- Save login sessions
- Save employee payroll objects
- Save game progress

Serializable Class

```
import java.io.Serializable;

class Student implements Serializable {
  int id;
  String name;
}
```

Serialization Example

```
import java.io.*;

class Student implements Serializable {
  int id;
  String name;

  Student(int id, String name) {
    this.id = id;
    this.name = name;
  }
}

public class SerializationDemo {
  public static void main(String[] args) throws
      IOException {
    Student s1 = new Student(101, "Prem");
```

Serialization Example

```
        ObjectOutputStream oos =
        new ObjectOutputStream(new
            FileOutputStream("student.obj"));
        oos.writeObject(s1);
        oos.close();

        System.out.println("Object serialized
            successfully");
    }
}
```

Deserialization

Definition

Reading the saved object back from the file into the Java program.

Deserialization Example

```
import java.io.*;

class Student implements Serializable {
  int id;
  String name;

  Student(int id, String name) {
    this.id = id;
    this.name = name;
  }
}

public class DeserializationDemo {
  public static void main(String[] args)
  throws IOException, ClassNotFoundException {
```

Deserialization Example

```
        ObjectInputStream ois =
        new ObjectInputStream(new
            FileInputStream("student.obj"));

        Student s = (Student) ois.readObject();
        ois.close();

        System.out.println("ID: " + s.id);
        System.out.println("Name: " + s.name);
    }
}
```

Basic Problem Statements: Serialization

1. Serialize a Student object
2. Deserialize and print student details
3. Save employee object into a file
4. Save patient record object

Important Methods

close()

Closes the stream and releases resources.

flush()

Forces buffered data to be written immediately.

throws IOException

Used because file operations may fail.

FileInputStream vs FileReader

FileInputStream	**FileReader**
Reads bytes	Reads characters
Best for binary/raw files	Best for text files
Example: image, PDF	Example: txt, java file

FileOutputStream vs FileWriter

FileOutputStream	**FileWriter**
Writes bytes	Writes characters
Binary/raw data	Text data

Buffered Streams vs Normal Streams

Normal Stream	**Buffered Stream**
Slower	Faster
Direct file access often	Uses memory buffer

Serialization vs Deserialization

Serialization	**Deserialization**
Object $\rightarrow$ File	File $\rightarrow$ Object
Save object	Restore object

When to Use What?

- **Text file** → FileReader / FileWriter
- **Binary or raw data** → FileInputStream / FileOutputStream
- **Faster operations** → Buffered Streams
- **Primitive data types** → Data Streams
- **Formatted output** → PrintStream
- **Save complete object** → Serialization

Mini Lab / Mini Project

Student Record Management System

Students can build a program that:

- Creates a file
- Writes student details
- Reads student details
- Stores marks using Data Streams
- Prints formatted result using PrintStream
- Serializes a Student object

Quick Viva Questions

1. What is I/O in Java?
2. What is the difference between byte stream and character stream?
3. What is a File object?
4. Why do we use buffering?
5. What is the purpose of DataInputStream?
6. What is Serialization?
7. What is the role of `flush()`?

Conclusion

Key Idea

Variables are temporary, but files provide permanent storage.

Real-world systems need files to save:

- Student records
- Hospital data
- Banking transactions
- User login details

Files and Streams are the bridge between memory and storage.

Thank You!

Stay Connected

Premanand S

Email:	premanand.s@vit.ac.in
Phone:	+91-7358679961
LinkedIn:	linkedin.com/in/premsanand
Instagram:	instagram.com/premsanand
WhatsApp Channel:	anandsDataX
Google Scholar:	Google Scholar Profile
GitHub:	github.com/anandprems